

Personalization level

		Examples	Features
Non-personalized	Traditional advert	• New product out now!	• Relies on reach of advertiser • Oblivious to market
	Editorial	• List of top editorial picks • Essential items	• Ratings from domain experts • No signal from user base
	Popularity-based	• Popular/trending items • People who buy X also buy Y	• Good option for new users • Head-focused
Personalized	Semi-personalized	• Age, gender • Country, profession	• Inferable, privacy-preserving • Coarse indicators
	Personalized	• Purchase history • What your friends like	• Most accurate, hits tail items • Higher complexity

Interaction

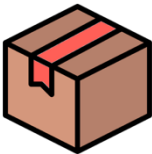
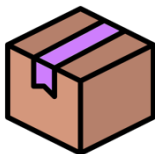
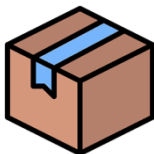
Explicit feedback ★★★★★

- Examples:
 - Unary rating (favorite)
 - Binary rating (like/dislike)
 - Numeric rating (stars)
- Pros
 - Direct measurement of preference
- Cons
 - Rare (requires extra effort from user)
 - Subjective (not comparable across users)

Implicit feedback 🔍

- Examples:
 - Views
 - Clicks
 - Bookmarks
 - Query
 - Scrolling
 - Start/stop/pause
- Pros
 - More abundant and varied
- Cons
 - Needs interpretation (e.g., time spent on platform not always good)
 - Not always a good proxy for preference

Rating matrix / utility matrix

		Items					
							
Users		4	4	4			
		4			3		
		2	1				5
			5		4		5

Usually:
#items >> #users
Sparse ratings

Objective

- Problem type
 - Prediction of unobserved user-item rating (matrix completion)
 - Ranking top-k items for a user (top-k recommendation)
- Metrics to maximize
 - Relevance/satisfaction
 - Coverage or item space
 - Novelty (coverage of item space from user perspective)
 - Serendipity (novel, with the right amount of “strange”)
 - Personalization (uniqueness of recommendations across users)

Algorithm families (for personalized recs)

- Content-based
 - Based on item attributes and their similarity
- Collaborative filtering
 - Model Based
 - Memory-based (neighborhood based)
 - User based: recommend items liked by similar people
 - Item based: recommend items similar to the items you liked
- Bandits
- Hybrid
 - Ensemble of different approaches

Ranking by simple aggregates

- Number of user-item interactions
 - Does not take ratings into account
- Average rating or upvote proportion
 - Do not take popularity into account
- Net upvotes (upvotes – downvotes)
 - Does not take controversy into account
- Full distribution of scores
 - Not directly applicable to ranking

[#JustinBieber](#) [#Baby](#) [#Remastered](#)

Justin Bieber - Baby (Official Music Video) ft. Ludacris

2,637,829,100 views • Feb 19, 2010

 **19M**  **38M**

Recommender Systems: The Textbook

by [Charu C. Aggarwal](#) | Mar 28, 2016

★★★★★ ~ 47

Rank by damped mean

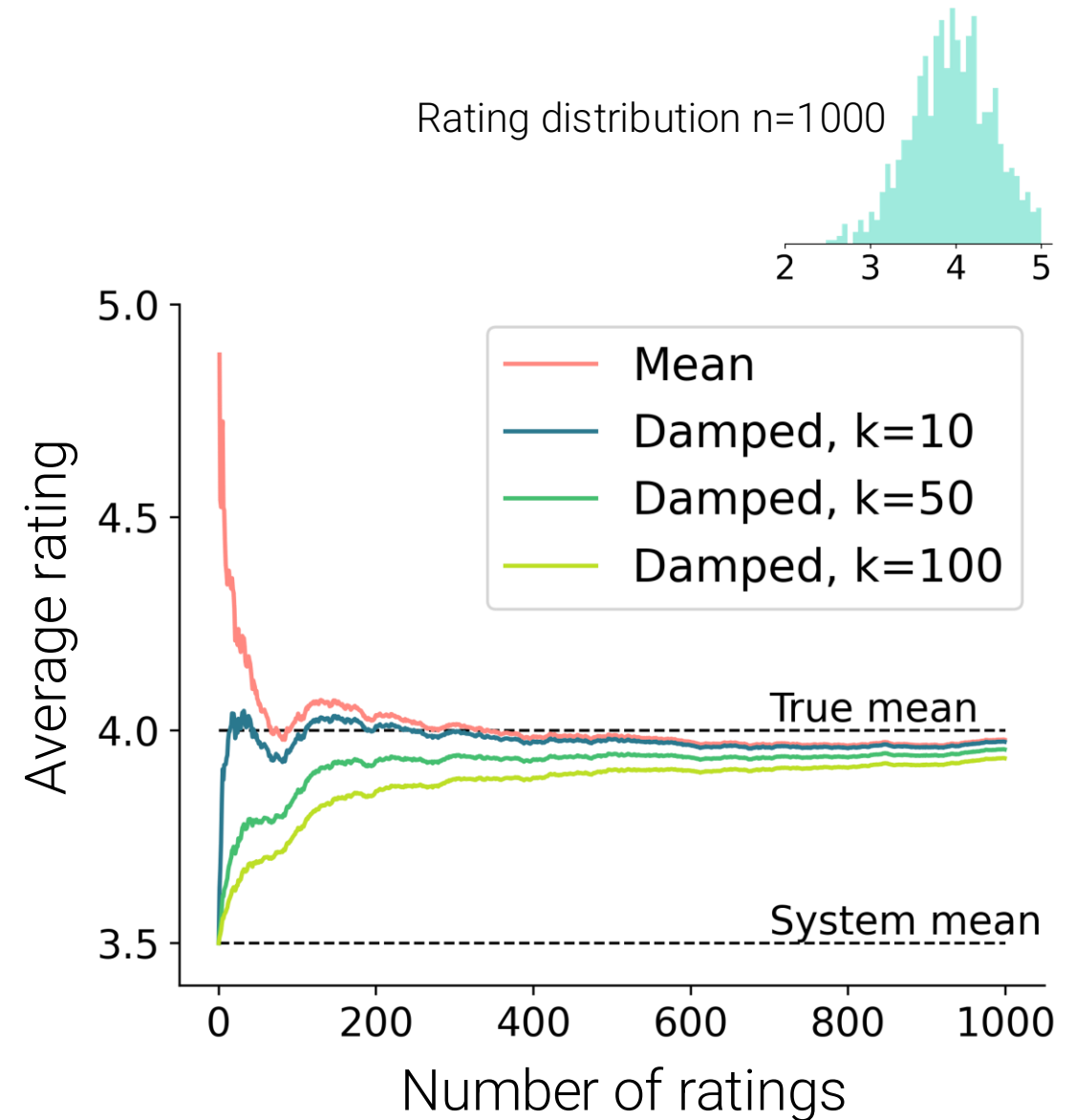
- Enforces a prior μ on the item rating
- Each item starts with k ratings equal to the prior

$$\bar{r}_i = \frac{\sum_u(r_{u,i}) + k\mu}{N + k}$$

Annotations for the equation:

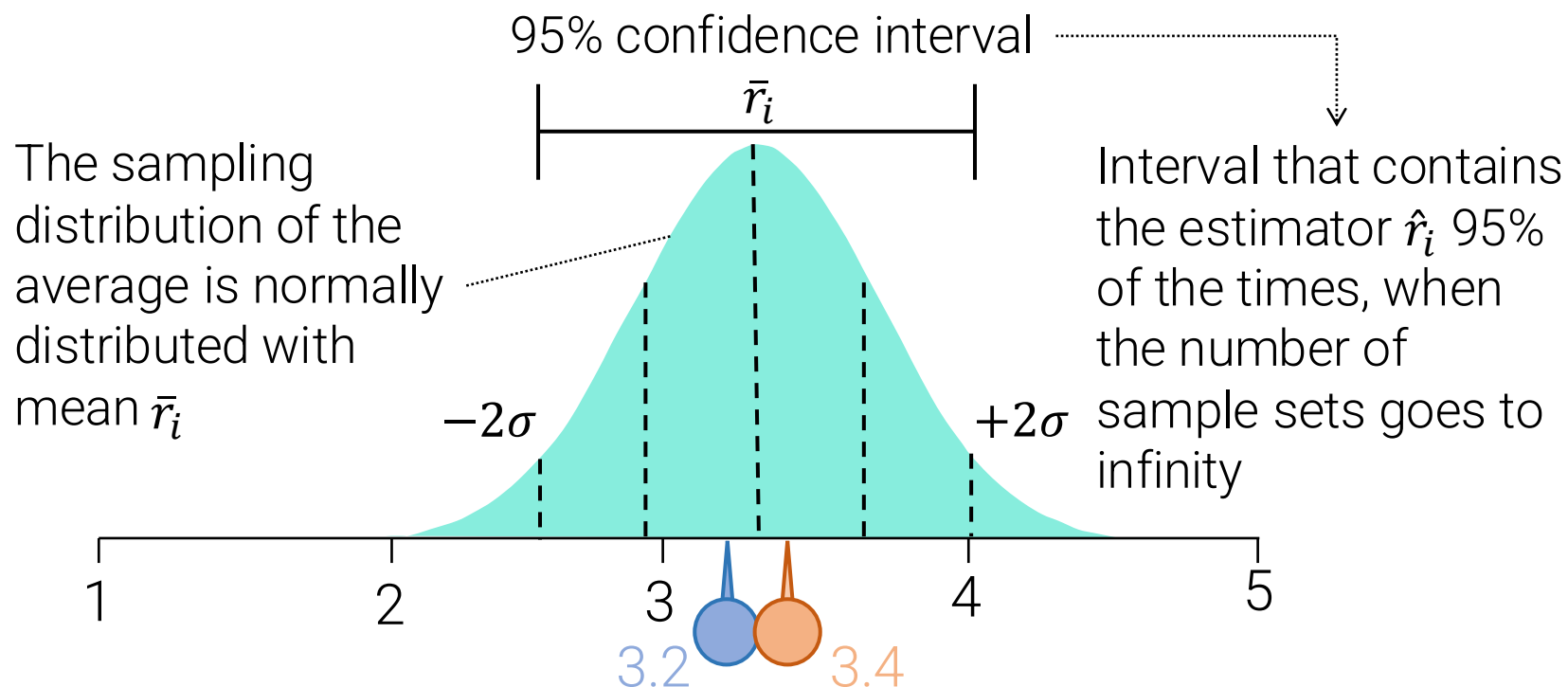
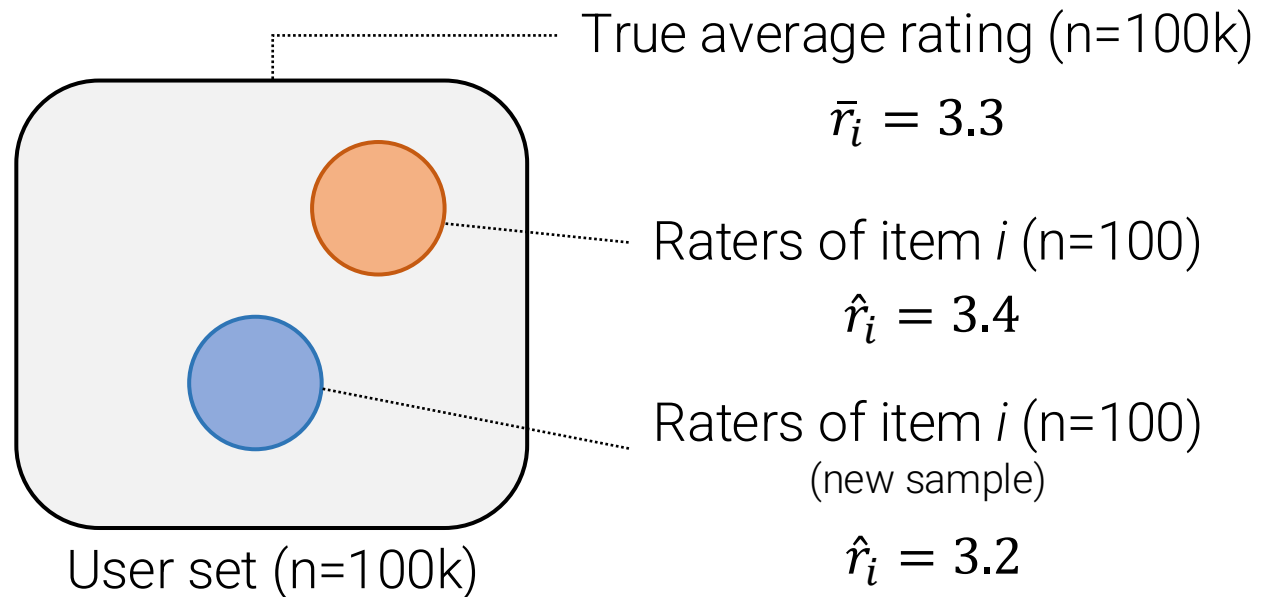
- μ : prior (global avg. rating)
- k : prior strength
- N : number of actual ratings for i

- New ratings skew the prior towards true mean



Confidence intervals

- Rank by lower bound of confidence interval
- Conservative approach



Sample std. deviation σ

Sample size n

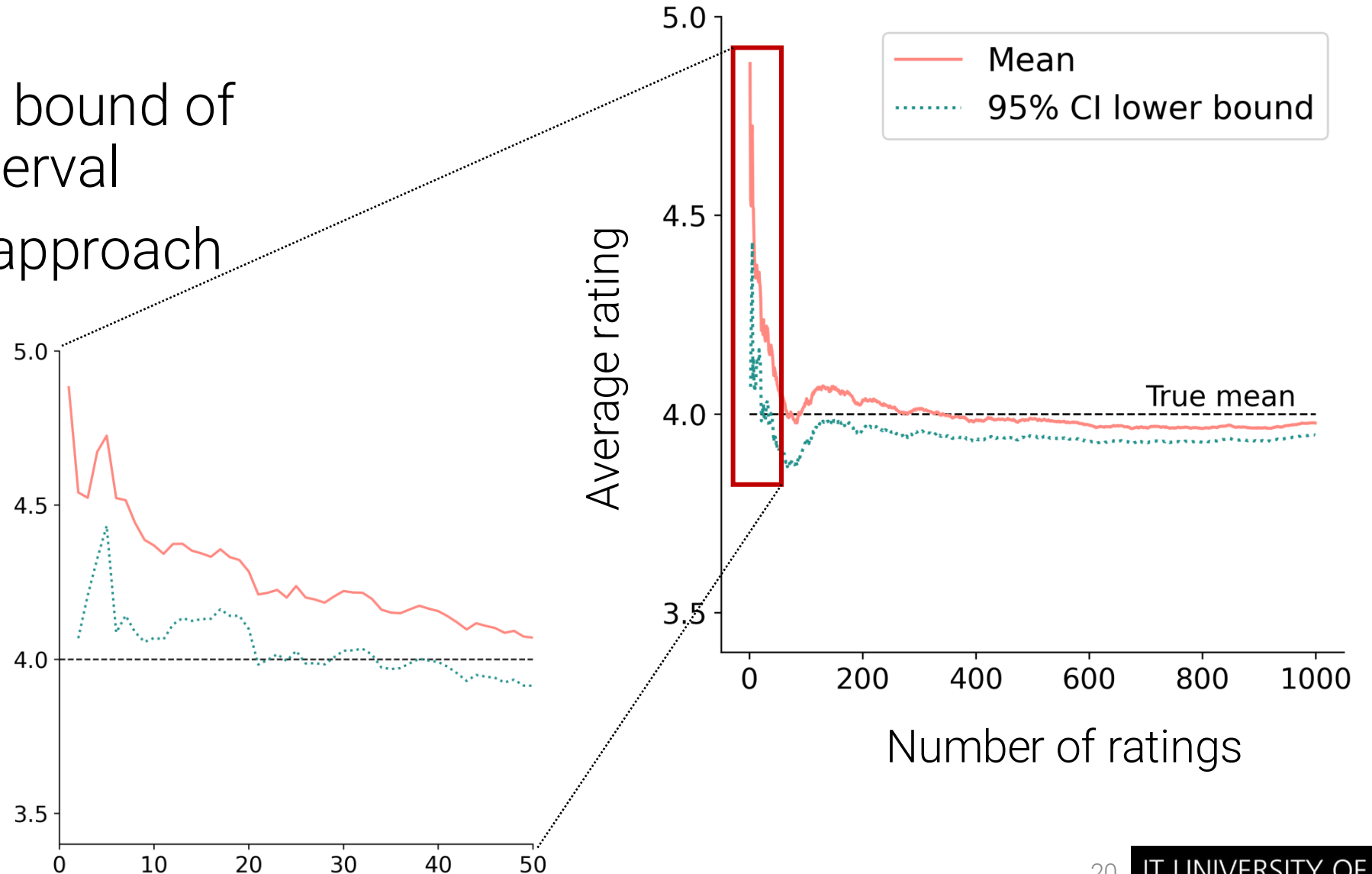
$$CI = \hat{r}_i \pm z_{1-\alpha/2} \cdot \frac{\sigma}{\sqrt{n}}$$

Critical value of normal distribution ($=1.96$ for 95%)

The equation block shows the formula for the confidence interval. A dotted line points from the text 'Sample std. deviation' to the symbol σ . Another dotted line points from the text 'Sample size' to the symbol n . A third dotted line points from the text 'Critical value of normal distribution ($=1.96$ for 95%)' to the symbol $z_{1-\alpha/2}$.

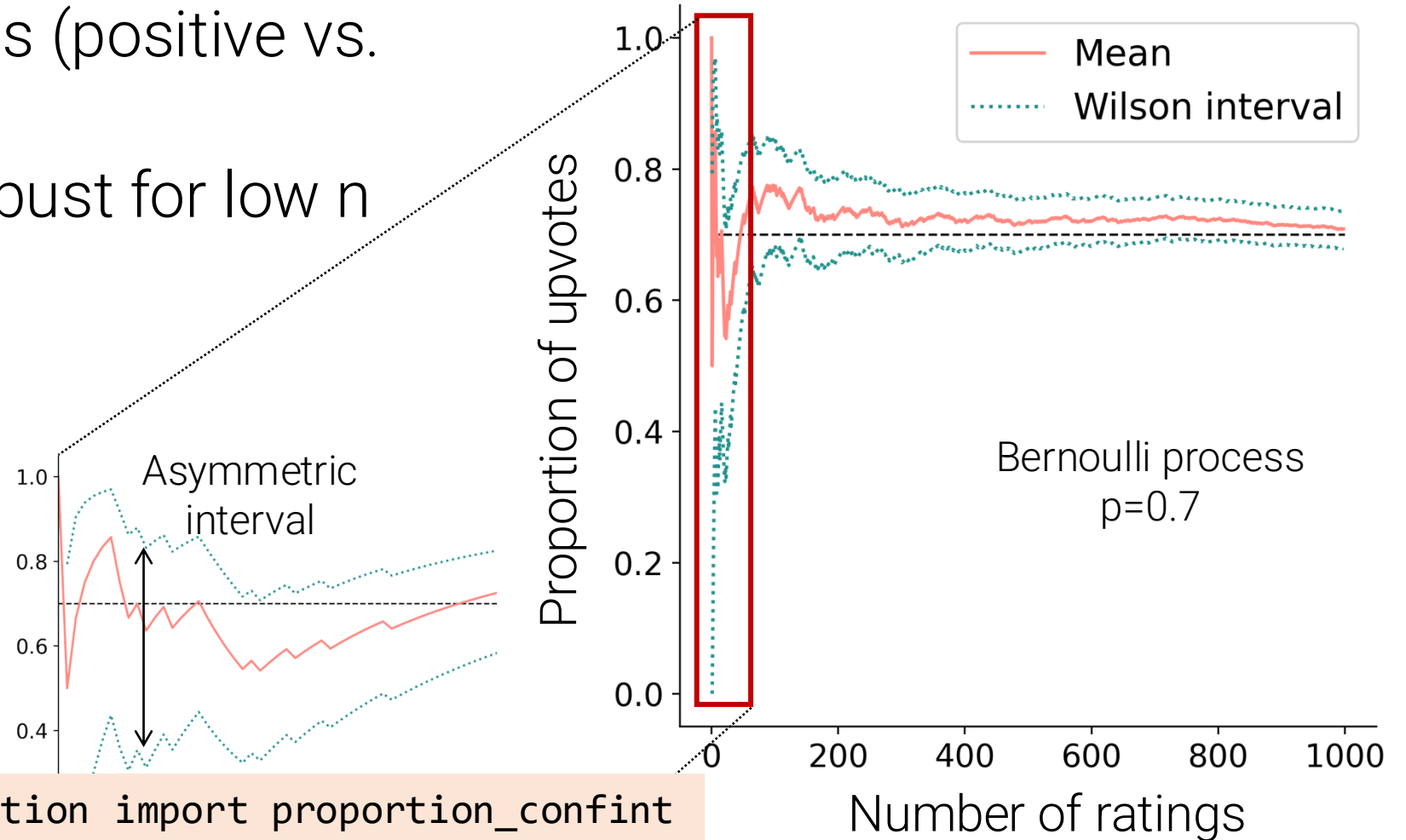
Rank by low confidence bound

- Rank by lower bound of confidence interval
- Conservative approach



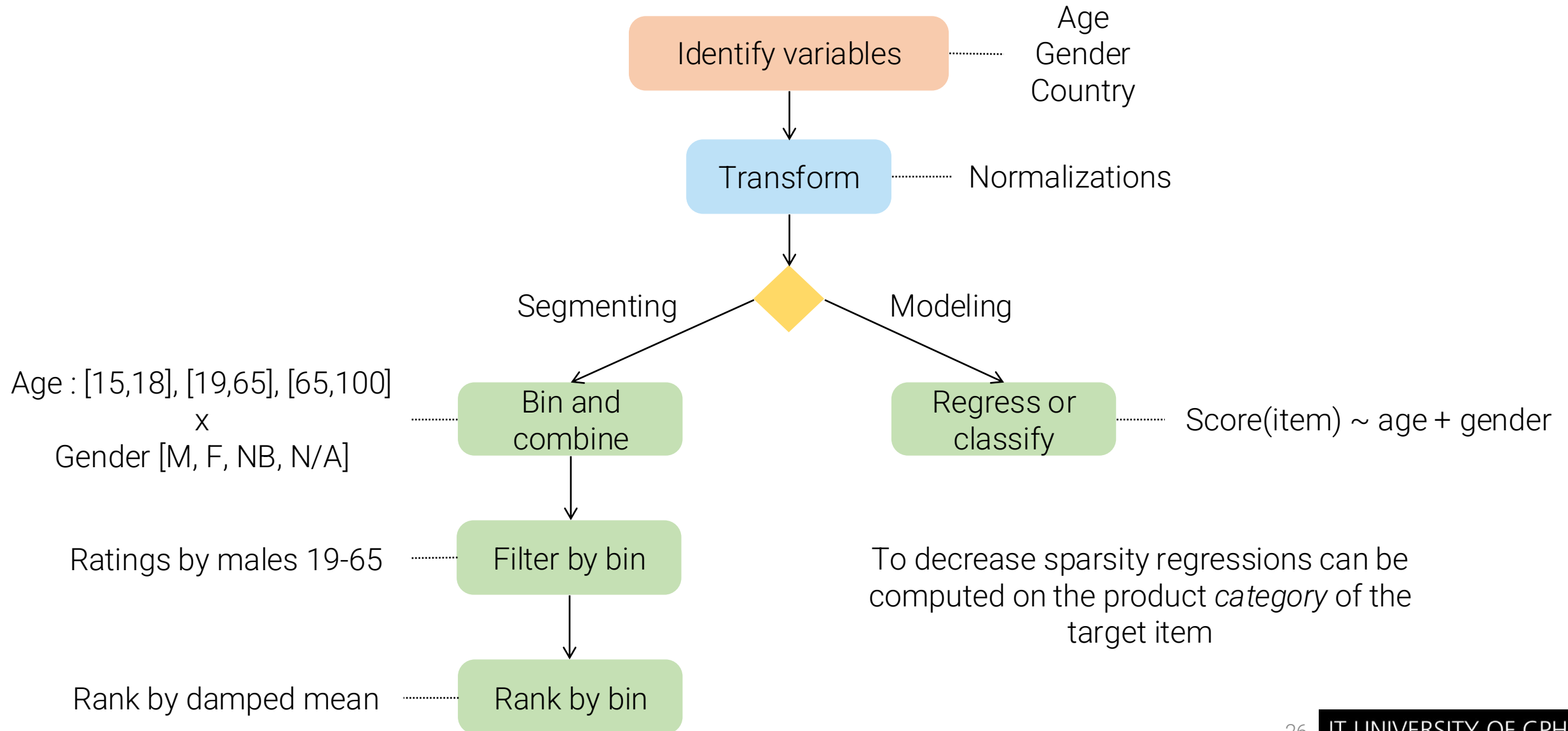
Wilson score confidence interval

- Used for proportions (positive vs. negative ratings)
- Asymmetric and robust for low n

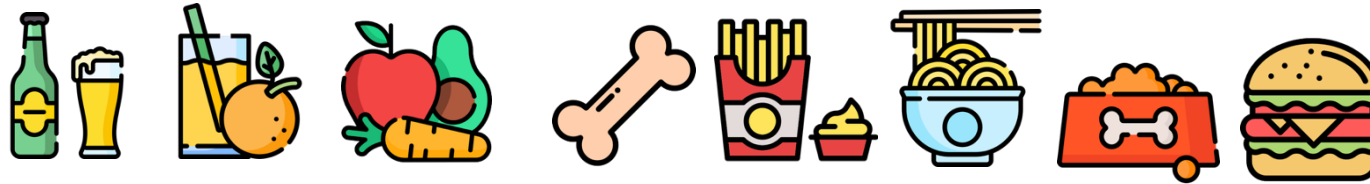


```
from statsmodels.stats.proportion import proportion_confint
proportion_confint(upvotes, total_votes,
                    alpha=0.05,
                    method='wilson')
```

Typical semi-personalized recs workflow



Itemsets



1 2 3 4 5 6 7 8



A

X	X	X		X	X		
---	---	---	--	---	---	--	--



B

			X			X	X
--	--	--	---	--	--	---	---



C

X				X			X
---	--	--	--	---	--	--	---



D

		X		X	X		X
--	--	---	--	---	---	--	---

Association rule lift (l)

–also known as *interest* or
pointwise mutual information–

Probability increase of joint
purchase compared to
baseline probability assuming
independence

$$l(X, Y) = \frac{s(X \cup Y)}{s(X) \cdot s(Y)}$$

Equivalent to $p(X \cup Y) / (p(X) \cdot p(Y))$

$$l(\{3\}, \{6\}) = 0.5 / (0.5 \cdot 0.5) = 2$$

$l \ll 1$: X and Y are substitutes

$l \gg 1$: X and Y are associated

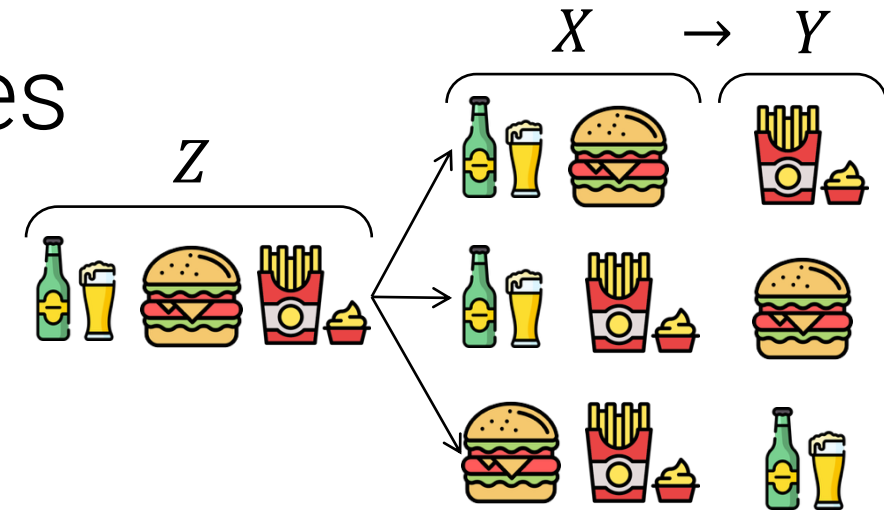
Finding frequent itemsets

- $s(Z) = p(Z) \geq s_{min}$ (support of Z is higher than a threshold)
 - The items in itemset Z co-occurred frequently
- Itemsets with low support are considered uninteresting (/statistically non-significant) and discarded
- A frequent itemset Z is then split into two itemsets X and Y , such that $X \rightarrow Y$ is an interesting association rule

Finding interesting association rules

- $Z = \{i_1, \dots, i_k\}; X = Z - \{i_m\}; Y = \{i_m\}$

- For all frequent itemsets, list all k possible rules with 1 consequent



- $c(X, Y) = p(Y|X) \geq c_{min}$. (association rule confidence is higher than a threshold)
 - Y occurred very frequently when X was purchased (directional, rule strength, used for filtering)
- $l(X, Y) = \frac{p(X \cup Y)}{p(X)p(Y)} \geq 1 + l_{min}$ (association rule lift is higher than a threshold)
 - The probability of co-occurrence is higher than an independence baseline (non-directional, used for filtering and sorting)

Algorithms to find frequent itemsets

- A-Priori
 - Classic
 - PCY, (& hash based variants)
 - SON
- FP-Growth

A-Priori summary

- At each step k
 - Generate candidates k -tuples by combining frequent $(k-1)$ -tuples
 - Count k -tuples in database
 - Obtain frequent k -tuples (tuples with frequency $\geq$ min support)
- In practice, frequent k -tuples with $k > 2$ are very rare (given a fixed minimum support)
- Enumerating all candidate pairs takes the most memory load

Apriori Python (apyori)

<https://pypi.org/project/apyori/>

```
from apyori import apriori
carts = [[0,1,2], [2,3], [0,3], ...] #list of carts
association_rules = apriori(carts, min_support=0.45,
                             min_confidence=0.2,
                             min_lift=3, min_length=2)

RelationRecord(items=frozenset({123, 999}), #frequent itemset
                support=0.00658,
                ordered_statistics=[
                    OrderedStatistic(
                        items_base=frozenset({999}), #antecedent
                        items_add=frozenset({123}), #consequent
                        confidence=0.24138913546152918,
                        lift=3.656822949209379)
                ])
)
```

APriori/FP-Growth Python (mlxtend)

```
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, fpgrowth
from mlxtend.frequent_patterns import association_rules

# encode dataset into a sparse [order x item] matrix
te = TransactionEncoder()
te_data = te.fit(carts).transform(carts, sparse=True)
df = pd.DataFrame.sparse.from_spmatrix(te_data, columns=te.columns_)

# calculate frequent itemsets (replace with fpgrowth ffor FP-Growth algorithm)
frequent_itemsets = apriori(df, min_support=0.3)

# calculate association rules by filtering on confidence or lift
rules = association_rules(frequent_itemsets, metric='confidence', min_threshold=0.5)
```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	leverage	conviction
0	(47209)	(13176)	0.066011	0.118028	0.019354	0.293199	2.484160	0.011563	1.247838
1	(21903, 13176)	(47209)	0.015722	0.066011	0.003793	0.241275	3.655095	0.002756	1.230999

MovieLens



- GroupLens is a research group specialized in RecSys
- Movielens is GroupLens' online service that collects movie ratings and provides recommendations
- 27M ratings (with timestamps) of 58K movies (with publication year) by 280K users until year 2018

<https://grouplens.org/datasets/movielens/>

MovieLens

movieId		title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy

Movies

userId	movieId	tag	timestamp
0	2	60756	funny
1	2	60756	Highly quotable
2	2	60756	will ferrell
3	2	89774	Boxing story
4	2	89774	MMA

Tags

userId	movieId	rating	timestamp
0	1	1	4.0
1	1	3	4.0
2	1	6	4.0
3	1	47	5.0
4	1	50	5.0

Ratings

Instacart



- Service for grocery delivery or pickup
- Open dataset
 - Year 2017
 - 3M grocery orders
 - 200K users
- Items grouped by orders

<https://tech.instacart.com/3-million-instacart-orders-open-sourced-d40d29ead6f2>

Instacart

	product_id	product_name	aisle_id	department_id
0	1	Chocolate Sandwich Cookies	61	19
1	2	All-Seasons Salt	104	13
2	3	Robust Golden Unsweetened Oolong Tea	94	7
3	4	Smart Ones Classic Favorites Mini Rigatoni Wit...	38	1
4	5	Green Chile Anytime Sauce	5	13

Products

	order_id	product_id	add_to_cart_order	reordered
0	2	33120	1	1
1	2	28985	2	1
2	2	9327	3	0
3	2	45918	4	1
4	2	30035	5	0

Carts

Exercise

Movielens

- Recommend 10 movies based on recent ratings
- Use techniques to make sure the recommendation is reliable

Instacart

- Calculate the number of frequent itemsets with varying levels for support
 - Try to guess what value of minimum support would be reasonable
- Calculate association rules and find the one whose subsequent item has the least support (the one more in the tail)